コンパイラ (9)

2026/10/30

属性文法

属性文法 (attribute grammar): 抽象構文木に属性を加えた文法

構文主導型変換への応用=コード生成に関する情報を属性として扱う

属性文法 $A_g = (G, \mathcal{A}, E)$

$G = (N, \Sigma, P, S)$ は CF 文法

$\mathcal{A}$: 属性の集合

$\mathcal{A} = \cup_{X \in N \cup \Sigma} \mathcal{A}(X)$. $a \in \mathcal{A}(X)$ のとき、$X.a$ と書く。

E: 意味規則の集合

$p \in P$ に対して以下のような規則を定める。

$$p\{X_i.a_i = f_i(X_{i_1}.a_{i_1}, \ldots, X_{i_n}.a_{i_n})\}$$

$$\begin{array}{ll} E \to E^{(1)}+E^{(2)} & \{E.val = E^{(1)}.val + E^{(2)}.val\} \\ E \to \mathtt{NUM} & \{E.val = \mathtt{NUM}.val\} \end{array}$$

属性の種類

合成 (synthesized) **属性**
右辺の属性から合成された左辺の属性:

相続 (inherited) **属性** (**「継承属性」とも言う**)
合成以外の属性

$$G = (\{T, L\}, \{\texttt{MAX}, \texttt{MIN}, \texttt{NUM}, \text{`}(\text{'}, \text{`})\text{'}, \text{`,'}\}, P, T)$$

$\mathcal{A}$:

	合成属性	相続属性
T	$T.val$	
L	$L.val$	$L.op$

$$P = \left\{ \begin{array}{ll} T \to \texttt{MIN}\ \text{`}(\text{'}\ L\ \text{`})\text{'} & \{L.op = \texttt{MIN}, T.val = L.val\} \\ T \to \texttt{MAX}\ \text{`}(\text{'}\ L\ \text{`})\text{'} & \{L.op = \texttt{MAX}, T.val = L.val\} \\ L \to \texttt{NUM} & \{L.val = \texttt{NUM}.val\} \\ L \to L^{(1)}\ \text{`,'}\ \texttt{NUM} & \left\{ \begin{array}{l} L.val = f(L.op, L^{(1)}.val, \texttt{NUM}.val), \\ L^{(1)}.op = L.op \end{array} \right\} \end{array} \right\}$$

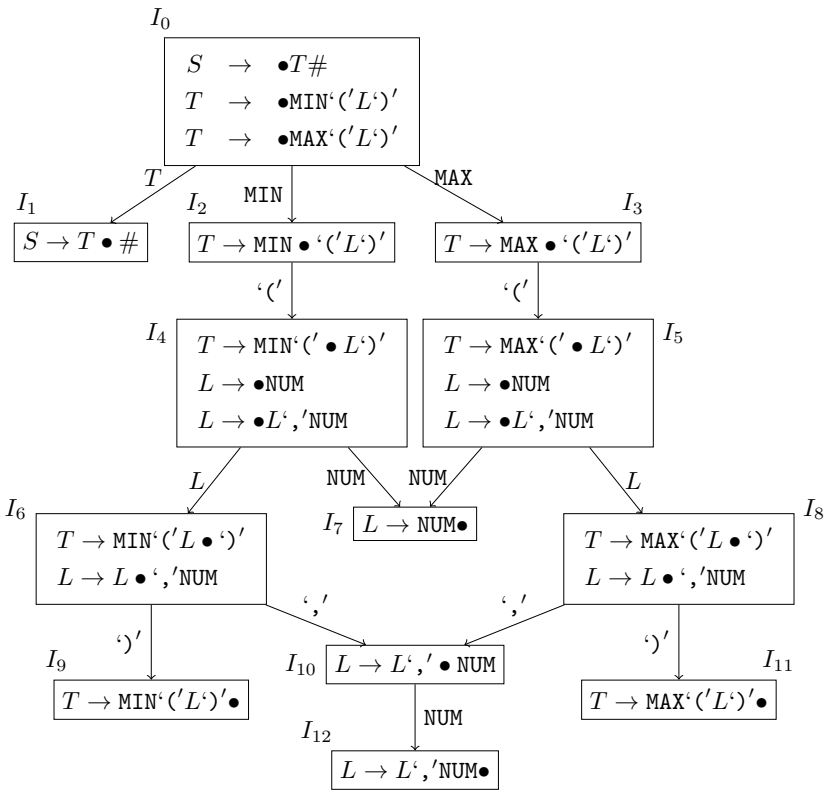
属性文法に基づくコード生成例

G に生成規則 $\{S \to T\#\}$ を加えた拡張文法に対する LR 解析

$$P = \left\{ \begin{array}{lll} 0: & S \to T\#, & 3: \quad L \to \texttt{NUM}, \\ 1: & T \to \texttt{MIN}\ \text{`}(\text{'}\ L\ \text{`})\text{'}, & 4: \quad L \to L\ \text{`,'}\ \texttt{NUM}, \\ 2: & T \to \texttt{MAX}\ \text{`}(\text{'}\ L\ \text{`})\text{'} & \end{array} \right\}$$

$\mathrm{Follow}_1(T) = \{\#\}$, $\mathrm{Follow}_1(L) = \{\text{`})\text{'}, \text{`,'}\}$

属性文法に基づくコード生成例



LR(0) 遷移系

	action							GOTO	
	MIN	MAX	NUM	(	)	,	#	T	L
I_0	s2	s3						1	
I_1							acc		
I_2				s4					
I_3				s5					
I_4			s7						6
I_5			s7						8
I_6					s9	s10			
I_7					r3	r3			
I_8					s11	s10			
I_9							r1		
I_{10}			s12						
I_{11}							r2		
I_{12}					r4	r4			

SLR(1) 構文解析表

属性文法に基づくコード生成例

	$(I_0,$	MIN(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2,$	(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4,$	3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4\texttt{3}I_7,$	,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	,5,7)#,	3)	$\%1.val = 3$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10},$	5,7)#,	3)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10}\texttt{5}I_{12},$	,7)#,	3)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	,7)#,	34)	$\%2.val = f(\%1.op, \%1.val, 5);$ $\%1.op = \%2.op$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10},$	7)#,	34)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10}\texttt{7}I_{12},$	)#,	34)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	)#,	344)	$\%3.val = f(\%2.op, \%2.val, 7);$ $\%2.op = \%3.op$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{)}I_9,$	#,	344)	
$\vdash$	$(I_0\,TI_1,$	#,	3441)	$\%3.op = min;\%4.val = \%3.val$
$\vdash$	$(acc,$	#,	34410)	

相続属性 (f の第一引数) が最後に決定される

$T.val = \%4.val$

属性文法に基づくコード生成例

	$(I_0,$	MIN(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2,$	(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4,$	3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4\texttt{3}I_7,$	,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	,5,7)#,	3)	$\%1.val = 3$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10},$	5,7)#,	3)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10}5I_{12},$	,7)#,	3)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	,7)#,	34)	$\%2.val = f(min, \%1.val, 5);$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10},$	7)#,	34)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{,}I_{10}7I_{12},$	)#,	34)	
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6,$	)#,	344)	$\%3.val = f(min, \%2.val, 7);$
$\vdash$	$(I_0\texttt{MIN}I_2\texttt{(}I_4LI_6\texttt{)}I_9,$	#,	344)	
$\vdash$	$(I_0\,TI_1,$	#,	3441)	$\%4.val = \%3.val$
$\vdash$	$(acc,$	#,	34410)	

相続属性 (f の第一引数) をバックパッチによって関数を書き換える

$T.val = \%4.val$

属性文法に基づくコード生成例

	$(l_0,$	MAX(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(l_0\texttt{MAX}l_3,$	(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5,$	3,5,7)#,	$\varepsilon)$	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5\texttt{3}l_7,$	,5,7)#,	$\varepsilon)$	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8,$	,5,7)#,	3)	$\%1.val = 3$
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8\texttt{,}l_{10},$	5,7)#,	3)	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8\texttt{,}l_{10}\texttt{5}l_{12},$	,7)#,	3)	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8,$	,7)#,	34)	$\%2.val = f(\%1.op, \%1.val, 5);$ $\%1.op = \%2.op$
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8\texttt{,}l_{10},$	7)#,	34)	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8\texttt{,}l_{10}\texttt{7}l_{12},$	)#,	34)	
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8,$	)#,	344)	$\%3.val = f(\%2.op, \%2.val, 7);$ $\%2.op = \%3.op$
$\vdash$	$(l_0\texttt{MAX}l_3\texttt{(}l_5Ll_8\texttt{)}l_{11},$	#,	344)	
$\vdash$	$(l_0\,Tl_1,$	#,	3442)	$\%3.op = max;\%4.val = \%3.val$
$\vdash$	$(acc,$	#,	34420)	

相続属性 (f の第一引数) が最後に決定される

$T.val = \%4.val$

属性文法に基づくコード生成例

	$(I_0,$	MAX(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MAX}I_3,$	(3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5,$	3,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5\texttt{3}I_7,$	,5,7)#,	$\varepsilon)$	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8,$	,5,7)#,	3)	$\%1.val = 3$
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8\texttt{,}I_{10},$	5,7)#,	3)	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8\texttt{,}I_{10}\texttt{5}I_{12},$	,7)#,	3)	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8,$	,7)#,	34)	$\%2.val = f(max, \%1.val, 5);$
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8\texttt{,}I_{10},$	7)#,	34)	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8\texttt{,}I_{10}\texttt{7}I_{12},$	)#,	34)	
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8,$	)#,	344)	$\%3.val = f(max, \%2.val, 7);$
$\vdash$	$(I_0\texttt{MAX}I_3\texttt{(}I_5LI_8\texttt{)}I_{11},$	#,	344)	
$\vdash$	$(I_0\,TI_1,$	#,	3442)	$\%4.val = \%3.val$
$\vdash$	$(acc,$	#,	34420)	

相続属性 (f の第一引数) をバックパッチによって関数を書き換える
$T.val = \%4.val$

LLVM による条件文コード

if B then S_1 else S_2

```
 %1 =  [B を計算するコード]
 br i1 %1, label %L1, label %L2
 L1:
   [S1 のコード]
 br label %L3
 L2:
   [S2 のコード]
 br label %L3
 L3:
```

L1,L2,L3 は実際に定義される前に参照される．

S_2 の後の%L3 へのジャンプは不要．

条件文における論理式

条件文における条件の評価：結果待ち / 短絡型

条件が副作用を持つ場合に違いが出る

`if (x>100 && (z=y)<30) { 文1 } else { 文2 }`

z=y の評価結果は y の値

z=y が実行されるか？

結果待ち＝評価される、短絡型＝x>100 のとき評価される

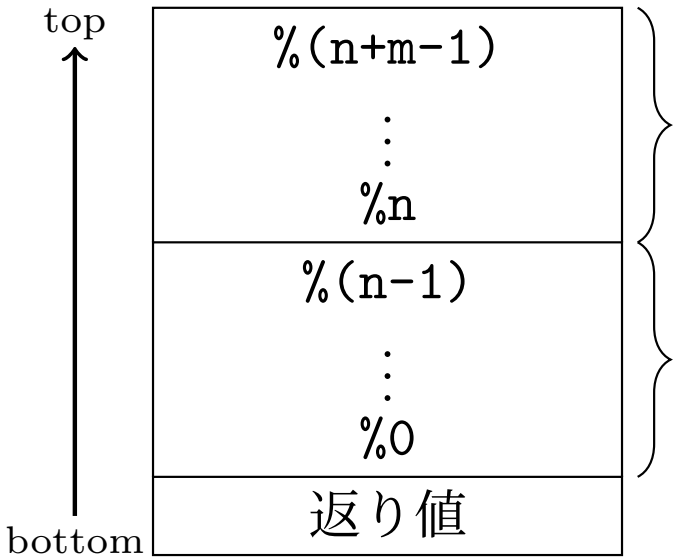
短絡型：　条件文の評価中に if 文を評価する．

結果待ち：　条件文の式を評価してから if を評価する．

短絡型の評価を行う場合が多い (C, Python)

Java の場合：&&, || が短絡型，&, | が結果待ち

関数引数・返り値と駆動レコード



局所領域 `f`$(x_1, \cdots, x_n)$ でスタック上に作成される駆動レコード（の一部）

引数領域 局所変数: $y_1, \cdots, y_m$

これに従った記号表を生成

値呼び： `x=f(`$e_1, \cdots, e_n$`)`

e_i を評価してその値を x_i の領域にコピー

参照呼び： `x=f(`$z_1, \cdots, z_n$`)`

z_i のアドレスを x_i にコピー

引数受渡し

引数=パラメータ (parameter)

2 種類の受渡し方法

値渡し：実引数の値を評価して仮引数に代入

参照渡し：実引数の値を評価してポインタを仮引数に代入

値呼び

```
void swap(x,y) {
    int t;
    t = x;
    x = y;
    y = t;
}
```

参照呼び

```
void swap(var x, var y) {
    int t;
    t = x; x = y; y=t;
}
```

値呼びの場合：全く効果なし。

入れ替えられた値は駆動レコード内で実行後破棄

参照呼びの場合：値が入れ替えられる。

swap 中の y=x, x=t の代入先は、実引数の領域

値呼びのコンパイル

%0: x の値，%1: y の値

```
void swap(x,y) {
    int t;
    t = x;
    x = y;
    y = t;
}
```

```
define void @swap(i32, i32) #0 {
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  %5 = alloca i32, align 4
  store i32 %0, i32* %3, align 4
  store i32 %1, i32* %4, align 4
  %6 = load i32, i32* %3, align 4
  store i32 %6, i32* %5, align 4
  %7 = load i32, i32* %4, align 4
  store i32 %7, i32* %3, align 4
  %8 = load i32, i32* %5, align 4
  store i32 %8, i32* %4, align 4
  ret void
}
```

参照呼びのコンパイル

%0, %1: x, y の値が格納されているアドレス

```
void swap(var x, var y) {
    int t;
    t = x; x = y; y=t;
}
```

```
define void @swap(i32*, i32*) #0 {
  %3 = alloca i32*, align 8
  %4 = alloca i32*, align 8
  %5 = alloca i32, align 4
  store i32* %0, i32** %3, align 8
  store i32* %1, i32** %4, align 8
  %6 = load i32*, i32** %3, align 8
  %7 = load i32, i32* %6, align 4
  store i32 %7, i32* %5, align 4
  %8 = load i32*, i32** %4, align 8
  %9 = load i32, i32* %8, align 4
  %10 = load i32*, i32** %3, align 8
  store i32 %9, i32* %10, align 4
  %11 = load i32, i32* %5, align 4
  %12 = load i32*, i32** %4, align 8
  store i32 %11, i32* %12, align 4
  ret void
}
```

関数定義/値呼び

```
int add(int x,int y) {
    return x+y;
}
```

```
define i32 @add(i32,i32) {
entry:
  % x = alloca i32
  % y = alloca i32
  store i32 % 0, i32* % x
  store i32 % 1, i32* % y
  % 2 = load i32, i32* % x
  % 3 = load i32, i32* % y
  % 4 = add i32 % 2, % 3
  ret i32 % 4
}
```

関数定義：

define ⟨ 型 ⟩ @⟨ 関数名 ⟩(引数型, · · · , 引数型)

i 番目の引数 %$(i-1)$ で参照

関数呼出（値呼び）：`%ret = call i32 @add(%2,%3), align 4`

コード最適化

コード最適化

実行の意味を変えることなく実行を高速化すること。

実行の意味：入力と出力
最適化の対象によって入力と出力は変わる。
プログラム解析が必要

- 機械独立なコード最適化
  =3 番地コードのレベルで実行を効率化
- 機械依存なコード最適化
  =対象マシンのアーキテクチャに依存
  (VLIW、マルチコアなど)

基本ブロック

最後以外分岐のない文の並び

```
prod := 0;
i := 0;
while (i < 20) begin
  prod := prod + a[i]*b[i];
  i := i + 1
end;
```

```
 prod = 0
 i = 0

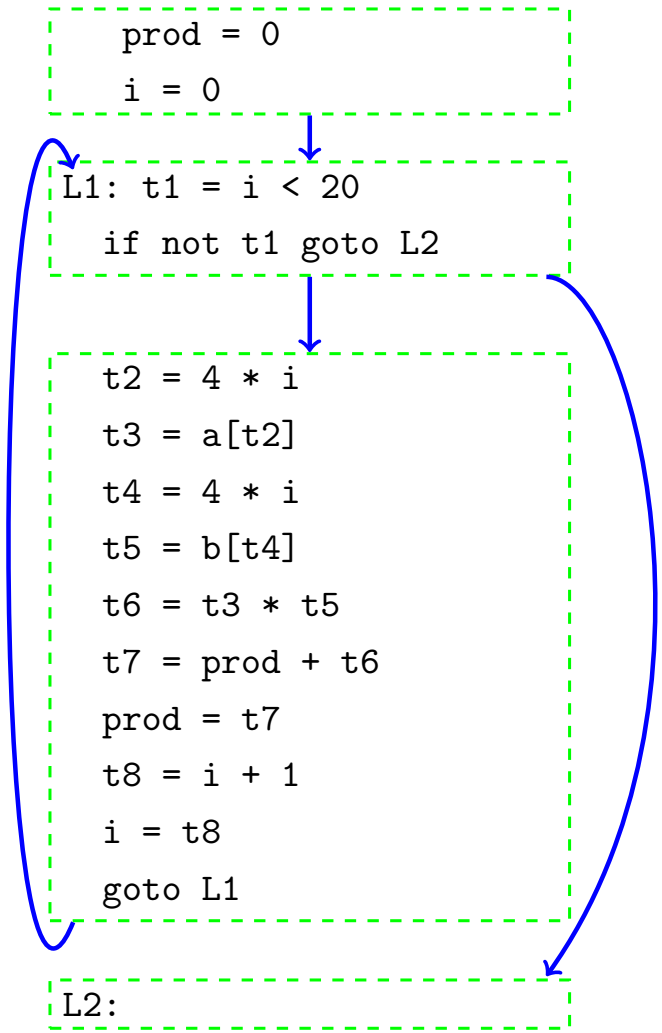
L1: t1 = i < 20
  if not t1 goto L2


  t2 = 4 * i
  t3 = a[t2]
  t4 = 4 * i
  t5 = b[t4]
  t6 = t3 * t5
  t7 = prod + t6
  prod = t7
  t8 = i + 1
  i = t8
  goto L1
```

基本ブロック

最後以外分岐のない文の並び

```
prod := 0;
i := 0;
while (i < 20) begin
  prod := prod + a[i]*b[i];
  i := i + 1
end;
```



局所最適化

基本ブロック内だけを見て最適化すること

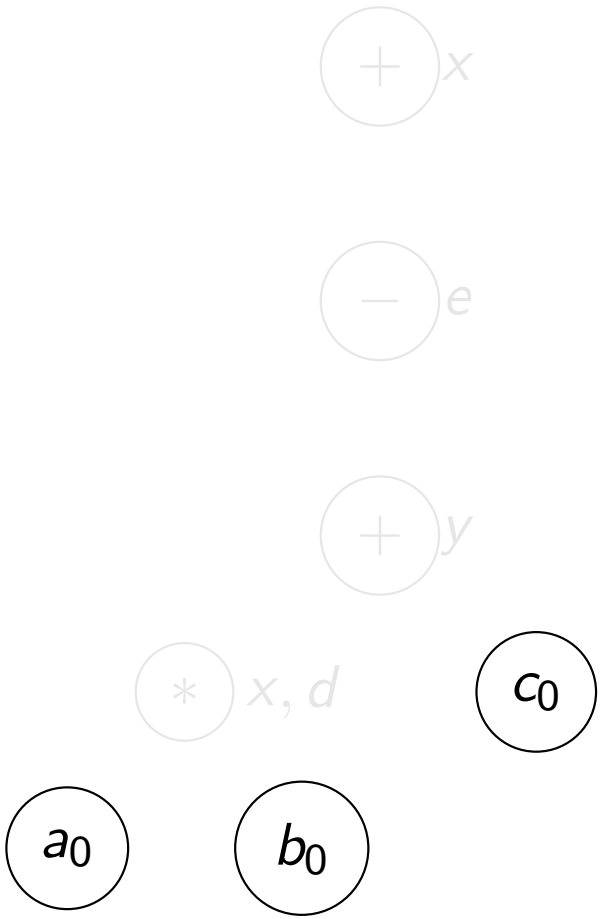
覗き穴最適化 (Peep-hole optimization)

- 定数伝播 (constant propagation)
  定数の計算、参照を定数そのものでおきかえる
- 共通部分式の削除 (common subexpression elimination)
  共通の部分式を抽出して結果を一時変数に保存するようにして、二回目以降計算しない。
- コピー伝播 (copy propagation)
  余分な変数の仲介を減らす。
  データ依存性を減らして命令の並列発行が可能になる場合がある。
- 不要コードの除去 (deadcode elimination)
  使われない変数に関する計算を消去する。
- 代数的性質の利用
  演算子が持つ代数的性質 (可換則、結合則、分配則、計算のためのコストなど)

DAG による局所最適化

DAG=Directed Acyclic Graph
閉路 (ループ) をもたない有向グラフ
基本ブロック内のデータ依存性を一意的に表現
DAG の構成アルゴリズム 6.6

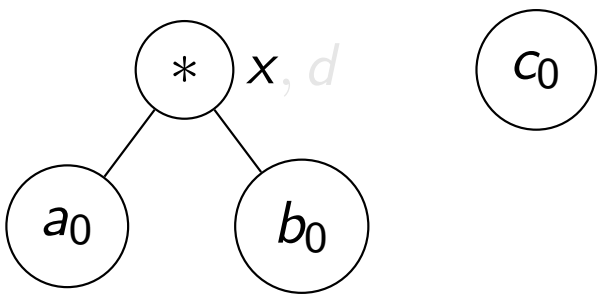
DAG の構成



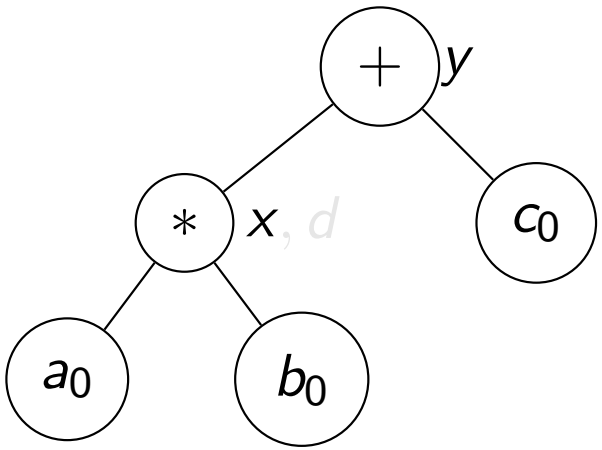

```
x=a*b
y=x+c
d=a*b
e=d-c
x=d+e
```

DAG の構成

x=a*b

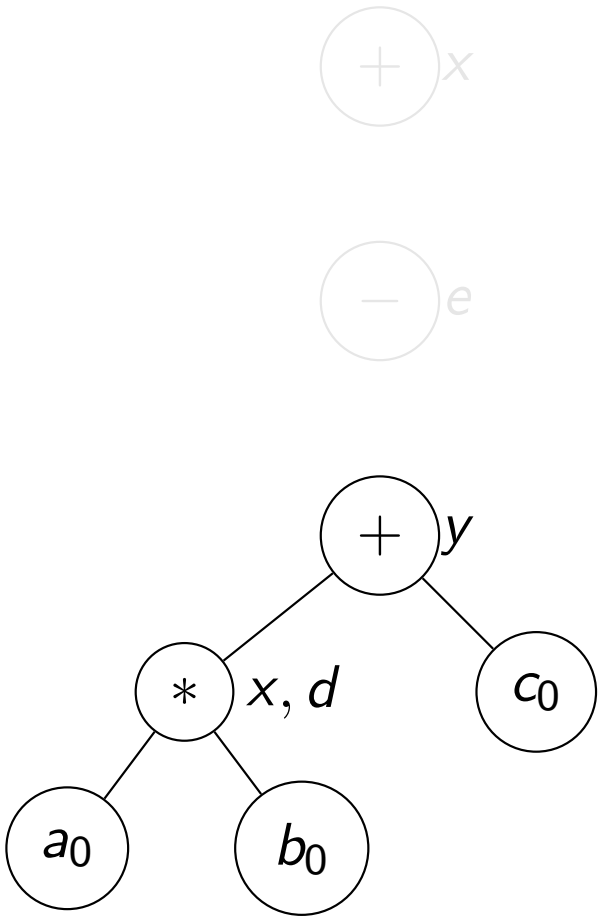


DAG の構成



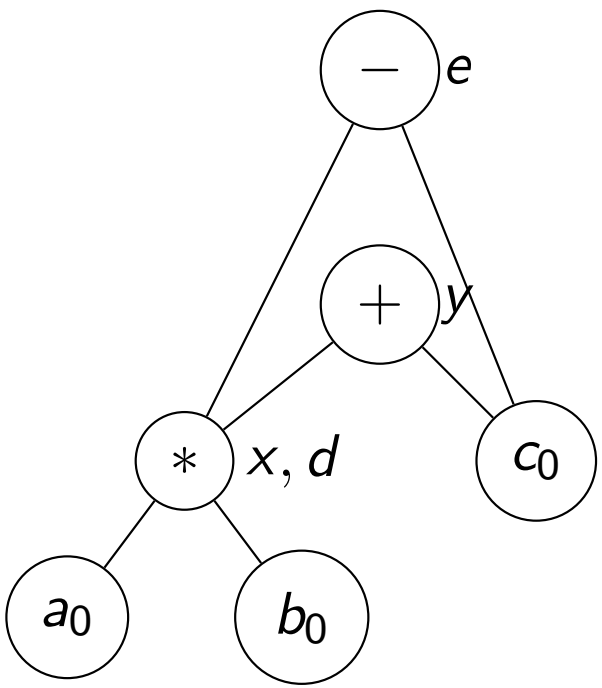

```
x=a*b
y=x+c
```

DAG の構成



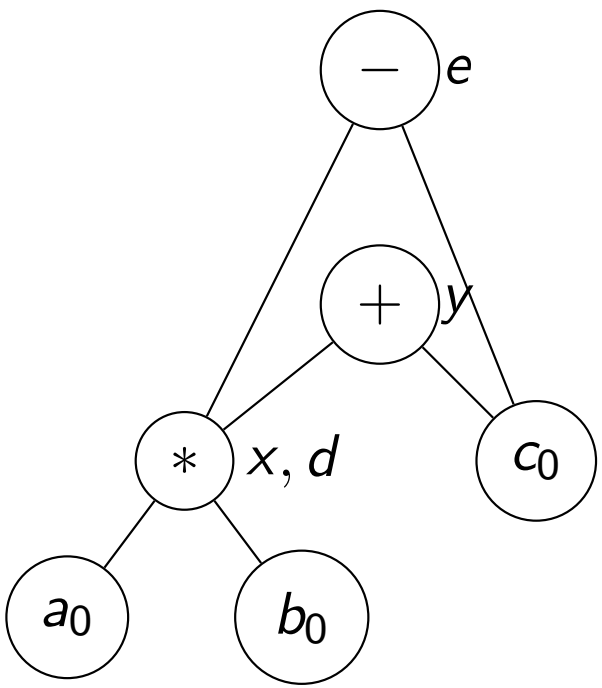

```
x=a*b
y=x+c
d=a*b
```

DAG の構成



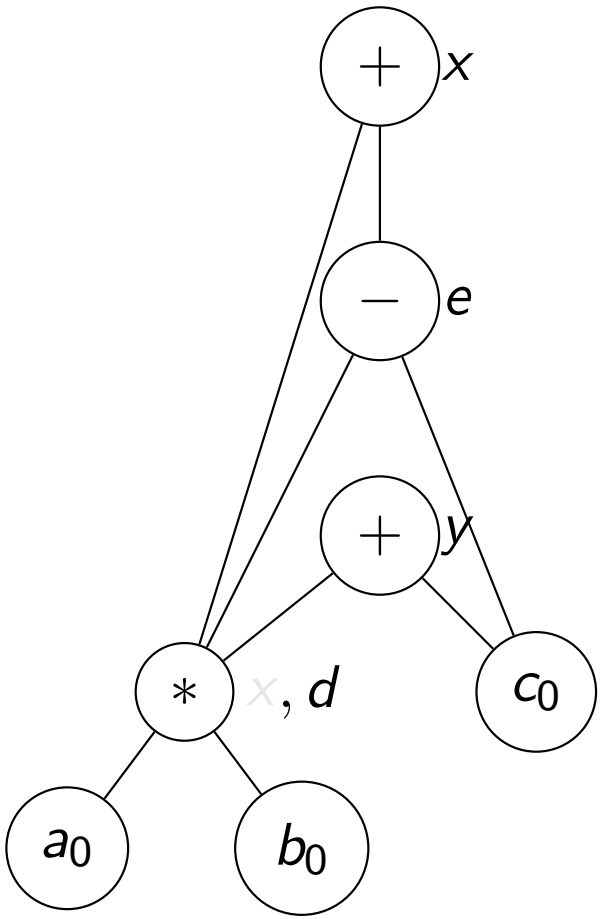

```
x=a*b
y=x+c
d=a*b
e=d-c
```

DAG の構成




```
x=a*b          x=a*b
y=x+c    ⇒     y=x+c
d=a*b          d=x
e=d-c          e=d-c
```

DAG の構成

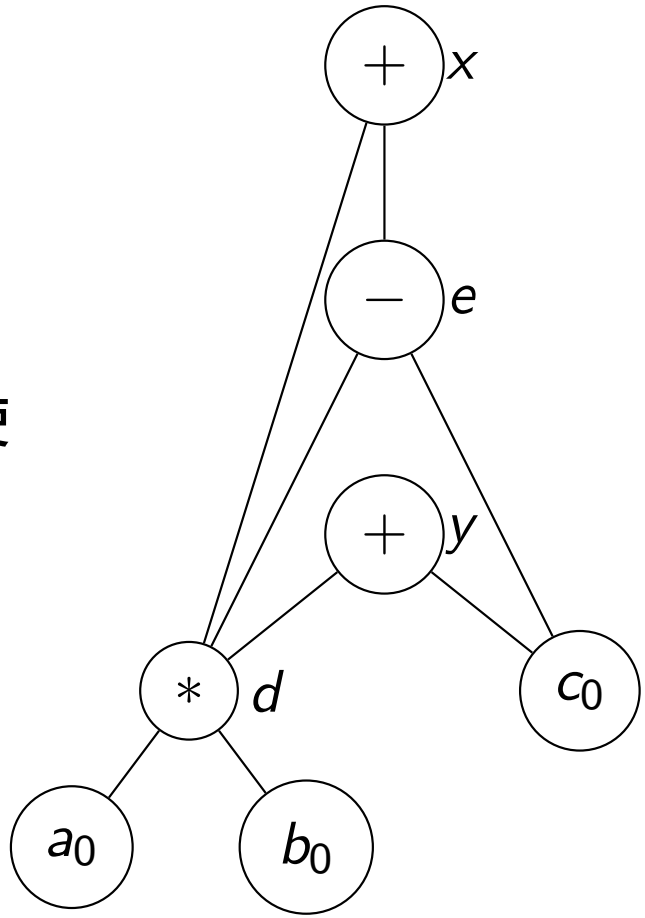



```
x=a*b
y=x+c
d=a*b
e=d-c
x=d+e
```

```
d=a*b
y=d+c
e=d-c
x=d+e
```

不要コードの消去

これ以降、x しか使用しないとすると

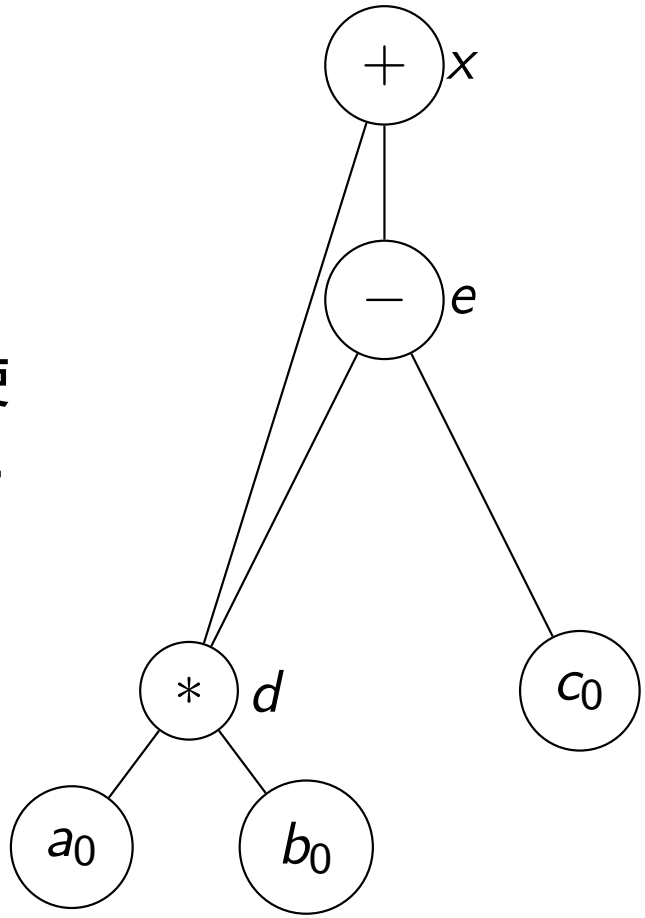



```
d=a*b
y=d+c
e=d-c
x=d+e
```

不要な変数を上からたどって消していく。
e,d は直接使わないが、x の計算に必要なので消去できない。

不要コードの消去

これ以降、x しか使用しないとすると
$\mathrm{y} = \mathrm{d} + \mathrm{c}$ は消去

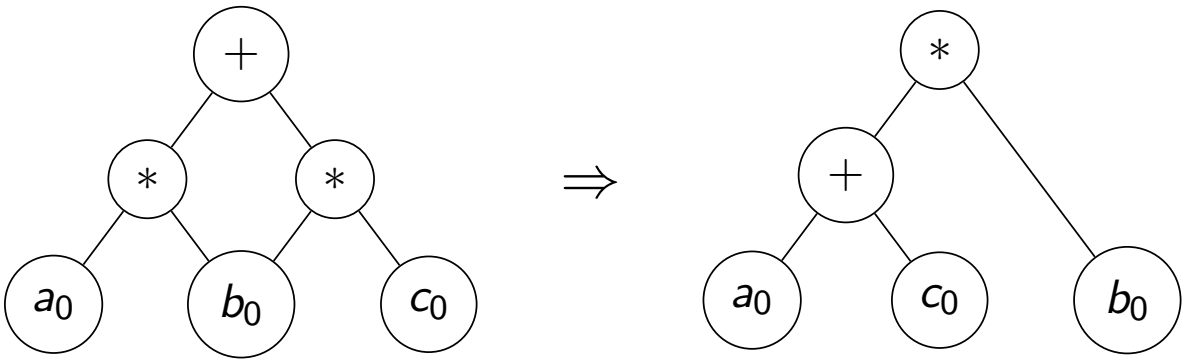



```
d=a*b

e=d-c
x=d+e
```

不要な変数を上からたどって消していく。
e,d は直接使わないが、x の計算に必要なので消去できない。

代数的性質




```
t1=a*b
t2=b*c
t3=t1+t2
```

```
t1=a+c
t2=t1*b
```

分配則=DAG 上でのパターンマッチング